

agement metrics, which can indirectly impact performance, such as bounce rates and session durations.

Use Cases: Track traffic, audience insights, user behavior, conversion tracking, e-commerce performance.

Core Web Vitals: While not explicitly tracked, user engagement metrics can reveal performance-related issues.

Cons: Data sampling may reduce accuracy, and real-time reports for custom metrics are limited.

2. Google Search Console

Google Search Console (GSC) is a crucial tool for tracking our website's performance in Google Search. It offers specific insights into Core Web Vitals, allowing us to monitor LCP, CLS, and INP directly. GSC is especially helpful for identifying and fixing SEO-related issues that affect site visibility.

Use Cases: Track search performance, monitor indexing, submit sitemaps, identify SEO issues, track Core Web Vitals.

Core Web Vitals: Provides direct tracking for LCP, CLS, and INP.

Cons: Limited historical data and data updates are delayed by a day or two.

3. ContentKing

ContentKing is an SEO monitoring tool that provides real-time tracking of our site's SEO health, including Core Web Vitals. It's ideal for catching issues before they impact rankings and tracking on-page changes over time.

Use Cases: Real-time SEO monitoring, change tracking, on-page audits, team collaboration, Core Web Vitals monitoring.

Core Web Vitals: Tracks LCP, CLS, and INP directly.

Cons: Primarily focused on SEO; doesn't provide detailed user behavior or traffic insights.

4. DebugBear

DebugBear is a performance monitoring tool tailored to track page speed and Core Web Vitals. It is designed for web developers and performance-focused users, providing deep insights into how various elements on the page affect load time and user experience.

Use Cases: Monitor Core Web Vitals, analyze page speed, simulate different network conditions, set performance alerts.

Core Web Vitals: Directly tracks LCP, CLS, and INP.

Cons: Lacks SEO and user traffic insights, limited integration with other tools.

CWV Testing Procedure

When you makes changes that might affect CWV, it is crucial to perform before and after tests to measure the impact of those changes. Here's the process:

1. Contact Point:

If you implement changes that could influence CWV metrics, contact Miracle or Hanif within the merge request (MR).

Clearly describe the nature of the change and why it might affect CWV.

2. Before and After Testing:

Miracle or Hanif will create a set of before and after tests to evaluate the impact of the changes on CWV.

Testing will be conducted on 3 URLs that are representative of the site's overall content

and structure.

The tests will be carried out using DebugBear, a reliable tool for monitoring CWV. Lighthouse in DevTools should not be used as it is known for discrepancies and may not provide consistent or accurate results.

The goal is to compare the CWV metrics of these URLs before and after the change.

3. Monitoring in Google Search Console:

For significant changes, it may take approximately one week after the change is merged for Google Search Console to reflect the impact on CWV based on real user data.

Monitoring these metrics in Google Search Console helps to confirm whether the changes have had the desired effect on actual site users.

Reporting and Follow-Up

After the testing is completed, results will be documented and shared with the team.

Any necessary adjustments or further optimizations based on the test results will be discussed and implemented accordingly.

Best Practices

Always initiate CWV testing for any changes that could impact loading performance, interactivity, or visual stability.

Utilize DebugBear for reliable monitoring, avoiding the use of Lighthouse in DevTools due to its inconsistencies.

Regularly monitor Google Search Console for any unexpected changes in CWV metrics.

By following these guidelines, we ensure that any changes made to the website are thoroughly tested for their impact on user experience, maintaining a high standard for CWV across all pages.

SECTION: marketing/digital-experience/engineering/data-dictionary.md

Digital definitions

Our goal is to ensure the consistency of data attribute keys and values for tagging the Marketing site. This will result in properly formatted event data getting added to the dataLayer and sent to Google Analytics.

Data Attribute Usage

Each link on the Marketing Site should have the two following data attributes:

- data-ga-name
- data-ga-location

An example of this usage for a free trial button might be as follows:

HTML

html

```
<a href="/free-trial/" data-ga-name="free trial" data-ga-location="header">Get free trial</a>
```

HAML

ruby

```
%a{href: '/free-trial', :data-ga-name => "free trial", :data-ga-location => "header"}
```

Or preferably:

ruby

```
= linkto "Get free trial", "/free-trial", :data-ga-name => "free trial", :data-ga-location  
=> "header"
```

What is the Name attribute?

The name should be, in English, what the button/link/input/etc says or points to.

An example for a page with a series of "Learn more" links that lead to different customer pages might have a name like goldman sachs link. Please note: we prefer lower case letters.

What is the Location attribute?

The purpose of the location attribute is to discern between multiple links with the same name on the same page. The location attribute is primarily for our top pages (see this link for our current top 50 pages). For the lower priority pages, simply having the name attribute should suffice.

| Key | Definition | Example |

| --- | ----- | ----- |

| nav | This link is found in the navigation of the page. | data-ga-location="nav" |

| header | This link is found in the header of a page. | data-ga-location="header" |

| body | If this link is the only one of its kind in the main body of the page (not the hero, not the nav/footer), you can use body. | data-ga-location="body" |

| {{section}} | If there are multiple links of this kind within the body, consider naming the location something descriptive about what section this particular link can be found under. Examples might be "features" or "benefits" | data-ga-location="features" |

| footer | This is for links in the footer of the page. The footer is the very bottom section with the purple background. | data-ga-location="footer" |

Utilities

To facilitate finding links contained in markdown inside files that do not highlight markdown, developers may use this regular expression:

text

```
(?:|[])\[(.?)\](.?)
```

Google Enhanced Ecommerce

We use Google's enhanced ecommerce for tag manager to collect information across our web properties.

Our data needs to conform to four APIs:

1. Product impressions

1. Add product

1. Checkout

1. Purchases

The values we use in that API are described below by key/value pair, where arrays of each value are arrays of possible values. In practice, only one should be used:

- name: [Ultimate, Premium, Compute, Storage]

- id: [0001, 0002, 0003, 0004]

- price: [99, 19, 10, 60]

- brand: GitLab

- category: DevOps

- variant: [SaaS, Self-Managed]

- quantity: Integer

- affiliation: GitLab

- option: Visa

- revenue: "Total Price" eg 289.34

- tax:"Tax Amount" eg: 13.45

- shipping:0

- coupon: "Leave Empty"

So an example product object might look like this:

```
js
{
  name: 'Ultimate',
  id: '0001',
  price: '99',
  brand: 'GitLab',
  category: 'DevOps',
  variant: 'Saas',
  quantity: 1,
  affiliation: 'GitLab',
  option: 'Visa',
  revenue: '289.34', // The total price paid, after taxes, fees, etc.
  tax: '13.45',
  shipping: '0',
  coupon: null,
}
```

This specification was derived from the work Empirical Path put together.

SECTION: marketing/digital-experience/engineering/dexbot.md

The Digital Experience team has created a Slack application to automate important messages and alerts to the team, the app configuration can be found [here](#)

Collaborators

Currently this application is managed by the following members of the Digital Experience team:

- Nathan Dubord
- Lauren Barker
- Megan Filo
- Laura Duggan
- Mateo Penagos

Contentful integration

Currently the main use for this bot is to notify the team as soon as one of the main pages is changed in our CMS platform, this will send a message that contains the following information to the team:

- Entry Title
- User ID of the user who made the changes
- Update time
- Entry ID
- Entry version after the changes

and also includes a button to open the comparison with the previous version in the CMS

this integration is done using two webhooks:

a Contentful Webhook which has been configured to send a custom payload containing the entry information to our Slack webhook which will then trigger the message from slack bot, this message is then sent to the dex-alerts channel in Slack

Slack blocks

The message that is sent was made using the slack Block system, which uses pre-made slack UI components to structure the message and add interactivity in the form of inputs and buttons, this is built in the form of JSON, this is the payload being used for the CMS alerts message:

```
json
{
  "blocks": [
```

```
{
  "type": "header",
  "text": {
    "type": "plaintext",
    "text": "Monitored page content updated",
    "emoji": true
  }
},
{
  "type": "section",
  "text": {
    "type": "plaintext",
    "emoji": true,
    "text": "Entry title: {/payload/fields/title/en-US}"
  }
},
{
  "type": "divider"
},
{
  "type": "section",
  "text": {
    "type": "mrkdown",
    "text": "Updated by user: {/payload/sys/updatedBy/sys/id} \n\n Update time:
{/payload/sys/updatedAt} \n\n Entry ID: {/payload/sys/id} \n\n Version:
{/payload/sys/revision}"
  },
  "accessory": {
    "type": "image",
    "imageUrl": "https://assets.stickpng.com/images/5a81af7d9123fa7bcc9b0793.png",
    "alttext": "calendar thumbnail"
  }
},
{
  "type": "divider"
},
{
  "type": "section",
  "text": {
    "type": "mrkdown",
    "text": "View changes in Contentful"
  },
  "accessory": {
    "type": "button",
    "text": {
      "type": "plaintext",
      "text": "View changes",
      "emoji": true
    }
  },
  "url": "https://app.contentful.com/spaces/{/payload/sys/space/sys/id}/entries/{/payload
```

```
/sys/id}/compare"
    }
  }
]
}
```

SECTION: marketing/digital-experience/engineering/engineering-ab-tests.md

Overview

An A/B test is when we release two versions of a page and compare how well they perform (comparing version A to version B). When running an experiment, we are testing a hypothesis using a control variant and a test variant, similar to how one typically employs the scientific method.

We currently use LaunchDarkly to control whether or not a test is showing, at what percentage, and gather metrics about a test's performance. Within LaunchDarkly, you can create events that fire when a user does something. For our case, our most common example would be a click. We also push the experiment ID into the Google Analytics dataLayer so we know what version of the page the user viewed.

```
js
window.dataLayer = window.dataLayer || [];

dataLayer.push({
  'event': 'launchDarklyExperiment',
  'launchDarklyExperimentName': 'name of experiment',
  'launchDarklyExperimentId': '0 or 1' //0 = control, 1 = variant
});
```

This project is for the ideation and refinement of AB Tests to be conducted on GitLab's Marketing site. Anyone can contribute an AB testing issue.

What is a feature flag

Below are some resources to learn more about feature flags. At a high level, a feature flag is an if-else wrapper around code that can be enabled, disabled, or served at a certain percentage to a certain group. This is controlled via a dashboard toggle, changing the production interface on-the-fly without having to wait for a release to change something.

An introduction to feature flagging software
Feature flag use cases

How we run AB test

Our AB tests include two files, the control and the test variant. Both exist on the page in the HTML DOM at the same time, but are hidden by default on page load. The javascript SDK will

return which version of the experiment should be shown. For each test, we use the following process:

1. Test Candidates are validated by modelling out the potential lift on an annualized basis.
1. AB test candidate with the highest annualized lift (lift scoped specifically to the action/improvement we're measuring) is run as an AB test.
1. AB test complete, data analysis on results.
1. The winner goes live at 100%.
1. Saves (tests that did not deliver as expected) are documented.

A/B Testing Schedule

Planned A/B Tests

Issue	Test Length	Status
-----	-----	-----
Homepage Featured Blocks vs Carousel		scheduled

Completed A/B Tests & Results

Issue	Variant A	Variant B	Winner
-----	-----	-----	-----
Remove 'register' from the navigation (top right) on about.gitlab.com	Variant A Image	Variant B Image	Variant B
Add borders around pricing tiers on /pricing based on DemandBase data	Variant A Image	Variant B Image	Variant B
Update Homepage Sub Copy	Variant A Image	Variant B Image	Variant B
Pricing Page: Free SaaS Trial + Self Managed Install CTAs	Variant A Image	Variant B Image	Variant B

How do we engineer tests

Running test on the Buyer's Experience repository

Developer notes on running an experiment: <https://gitlab.com/gitlab-com/marketing/digital-experience/buyer-experience/-/blob/main/docs/plugins.md>

Example MR for an AB test: <https://gitlab.com/gitlab-com/marketing/digital-experience/buyer-experience/-/mergerequests/258>

Video Explanation: <https://www.youtube.com/watch?v=F1tICZgYUqQ>

Running test on the www repository

This can be overridden by optional URL parameters as exhibited in the codepaths section below.

Example merge request for an AB test

Tutorials for implementing AB test in the www-gitlab-com project

Active

VIDEO: Part 1 of 3, explaining code paths and how to develop an AB test of the pricing page
VIDEO: Part 2 of 3, explaining how our feature flag vendor's system works and how to verify that the AB test is working

VIDEO: Part 3 of 3, explaining how to decommission a feature flag and set up a new control experience

DEPRECATED: Outdated Google doc and video explainer outlining the system. Code and structure has changed a lot since this video was made.

Additional Notes

Previous solution: run-experiments.js from www: <https://gitlab.com/gitlab-com/www-gitlab-com/-/blob/master/source/javascripts/run-experiment.js>

Originally tried to bring this file over and integrate it as a nuxt plugin in this MR:

<https://gitlab.com/gitlab-com/marketing/digital-experience/buyer-experience/-/mergerequests/224/diffs8f8555baf29e37a194c0d155b0001e82de21ddb>

Our buyer's experience repository is a JS pre-rendered site with TS support, so this would be a good candidate for interacting with their SDK TS directly:

<https://docs.launchdarkly.com/sdk/client-side/javascript/getting-started>

Initialize client, helper function to evaluate feature flag value: <https://gitlab.com/gitlab-com/marketing/digital-experience/buyer-experience/-/blob/main/plugins/launchdarkly.ts>

Higher order component with control and experiment slots: <https://gitlab.com/gitlab-com/marketing/digital-experience/buyer-experience/-/blob/main/components/launch-darkly.vue>

Why did we choose LaunchDarkly

Because the marketing website about.gitlab has no dynamic server, we needed a solution that could be performant and implemented using a javascript SDK. In addition to that, we needed a solution that was able to attach metrics.

Below are some links with information on the history of the decision:

GitLab marketing's use of LaunchDarkly

Why use a SaaS tool

Taking a step back when defining requirements

Epic for the original implementation

Original proof of concept

SECTION: marketing/digital-experience/engineering/engineering-gitlab-product.md

Overview

From time to time, our team has objectives that require us to collaborate on the GitLab product. The group we have most collaborated in the past has been the Growth team, though engineers should validate what stage they should contact. They should be informed and consulted with any changes that affect the GitLab product, ideally doing so before we begin any work. This ensures that we are aligned with their goals and can leverage their expertise in the GitLab product. Remember that the Digital Experience team's work on the GitLab product should align with our team's OKRs and the overall company strategy.

When working on the GitLab product, it's important to follow the established contribution guidelines and code review process. This includes:

1. Writing clear commit messages
2. Opening a merge request with a detailed description
3. Addressing any feedback from reviewers

Additionally, make sure to:

- Write and update tests as necessary
- Follow GitLab's coding standards
- Update documentation if your changes affect user-facing features

Collaboration Process

When collaborating on the GitLab product, follow these steps to ensure smooth communication and efficient work:

1. Identify the need: Determine if your objective requires changes to the GitLab product.
2. Consult with the right team: Before starting any work, reach out to the appropriate team to discuss your plans. This can be done through:
 - Scheduling a meeting
 - Posting in the appropriate Slack channel
 - Creating an issue in the relevant team's project board(if necessary)
3. Create a proposal: Draft a proposal that includes:
 - Define the scope outlining the changes you intend to make
 - The problem you're solving
 - The proposed solution
 - Any potential impacts on other parts of the product
 - Timeline and resource requirements
 - Who is committing to what work, and if any engineering assistance is necessary
4. Get approval: Obtain formal approval from the team and any other relevant stakeholders before proceeding with development.
5. Regular updates: Keep the right team informed of your progress through:
 - Prompt status updates
 - Sharing merge request links for review
 - Flagging any roadblocks or changes in scope
6. Final review: Before merging your changes, ensure the appropriate team has given their final approval.

Onboarding

There is an existing onboarding process for GitLab engineering. Digital Experience engineers need to complete a specific subset of that onboarding:

1. Git Workflow

- Make sure you still feel comfortable with day 5 of the general onboarding steps for git workflow

2. Engineering Department specific tasks

- Complete the Engineering Division/For Development Department specific tasks

3. Snowplow

- Complete the Snowplow onboarding tasks

4. GDK

1. You can get started with the GitLab Rails using the GDK here

2. Set it with one line installation which:

- Clones the GDK project into a new gitlab-development-kit directory in the current working directory.

- Installs asdf and necessary asdf plugins.

- Runs gdk install.

- Runs gdk start.

Note: First follow dependency installation instructions if necessary.

```
curl "https://gitlab.com/gitlab-org/gitlab-development-kit/-/raw/main/support/install" |  
bash
```

3. To browse your development server:

- Execute: cd gitlab-development-kit

- Execute: gdk start

4. Use the login credentials found here and learn more GDK commands

5. Read how to use GDK documentation

Slack Channels

Here are some slack channels to ask for help:

- gdk

- backend

- development

- frontend

SECTION: marketing/digital-experience/engineering/engineering-marketo.md

For more detailing instructions, visit Marketing Operation's handbook page related to snippets and forms

Snippets

Snippets are reusable chunks of content used in Marketo emails, landing pages, and templates. They can include a variety of content types.

Key Features

- Snippets can be used across multiple programs.
- Updates to snippets are automatically reflected everywhere the snippet is used.

Best Practices

- Double-check changes before saving, as updates are automatically propagated.
- Document all changes for reference and version control.

Templates

Templates provide the base layout and formatting for Marketo emails and landing pages.

Key Features

- Templates define where elements, including snippets, can be inserted.
- Updates to templates do not automatically reflect in previously created pages.

Best Practices

- Manually republish pages to reflect updates in templates.
- Be cautious when making changes that may impact the layout or appearance of your pages.
- Document all changes for reference and version control

Older Tutorials that are good, but somewhat stale

- DOC: Marketo workshop agenda, March 2021
- VIDEO: Marketo workshop, March 2021

SECTION: marketing/digital-experience/engineering/image-guidelines.md

Please note that deviation from these guidelines will increase production time.

The purpose of this page is to present guidelines for image file formats. The intended audiences are people who might not be familiar with design tools nor our unique requirements when working on items like about.gitLab.com, print collateral, and more.

We highly suggest you ask any partner/vendor/client for images that follow these guidelines. Examples of this include but are not limited to:

Partner pages
Event sponsors
Case studies
Comparison pages
Trusted by sections

File size limits

Our website has a 2MB file size limit. Please plan accordingly when providing image assets.

Dos and don'ts

Don't supply images inside of a google doc, powerpoint, etc. Please link to an image file directly, in a zip file, a google drive folder, or a git repository.

Logos

All logos should be provided in a vector format such as .SVG, .EPS, or .AI. An SVG is preferred.

SVG is a web-native format that any modern web browser can read.

Vector formats scale cleanly to devices of any size.

The SVG should be saved using presentation attributes rather than style attributes.

Presentation attributes are easier to work with when building websites.

All fonts & typography need to be outlined

Most users will not have a copy of any fonts used in a logo.

Please provide a monochromatic logo and a color logo.

Limiting color palettes is a way to reduce contrast-of-color in order to conserve attention-span.

Monochromatic logos are used in most places but we also use color logos in select locations. It helps to have both.

For dimensions, we require a width that is greater than or equal to its height.

Ideally an aspect ratio between 1:1 and 2:1 (width:height).

This is necessary to ensure that the logos will fit into specific locations on the website.

Artboards should be sized appropriately.

If for example a logo were 400px by 200px, please crop the artboard accordingly. Do not leave the logo on an artboard that's 1800px by 2600px.

It's preferable (but not required) that logos are lockups (logo + wordmark).

For example, the words "Pepsi" with their circle icon, as opposed to just the icon or just the name.

Lettermarks might be unrecognizable without a wordmark. Wordmarks alone can be unintelligible in other languages. Lettermarks can overlap with similar acronyms, potentially complicating internet searches.

It's preferable (but not required) that logos be legible at small sizes.

Please be aware that some elements of a logo might be too thin or small to read or discern at small sizes.

Photos

Photos usually start off as a high-quality lossless jpg before being optimized.

If we will be preparing a photo for you we ask that you supply us with a high quality file.

If you will be preparing the photo on your own, please ensure it is optimized per the above link and ask us to review the performance before merging.

If you are unsure what size the image needs to be, feel free to contact us for assistance.

All photos must have prior consent or licensing for use on our website.

We often use stock photos from Unsplash.

Graphics

In the rare case where a JPG or SVG aren't applicable, a full-color PNG with a transparent background is often preferred.

Animations

Animations are typically supplied via GIF but might be provided via SVG in certain circumstances. Due to limited browser and tooling support we do not accept WebP files at this time. Animations must conform to the file size limit of 2MB. Animations larger than 2MB might need to be converted into a video file such as MP4 with special arrangements and extra lead time for implementation.

SECTION: marketing/digital-experience/engineering/incident-response-playbook.md

Overview

This page offers insights into incidents occurring on the GitLab marketing site, delineates methods to assess severity levels, and outlines avenues for obtaining support.

At the outset, it's important to note the diverse composition of our marketing site, consisting of multiple projects. While all deployments converge into the same GCP bucket, they employ varying technologies for website generation.

1. The Marketing site is composed of multiple repositories: the blog, www, navigation, slippers and buyer experience.

1. www-gitlab-com, Buyer Experience, and the Blog generate pages during the build process and upload these artifacts to a single GCP bucket. Upon pipeline execution, all artifacts are consolidated within the /public directory on our GCP bucket.

What level is this incident?

The following are the questions to consider when determining incident severity:

1. What's the impact level of the marketing site outage?

1. Have you monitored the digital-experience-team or dex-alerts Slack channels for any ongoing incidents?

1. How extensive is the incident? It's crucial to assess beyond the number of affected individuals, considering:

- The total number of impacted users.
- The potential impact on various categories of our key stakeholders.
- Whether the incident affects significant customers or partners, regardless of the scale.

1. Are any affected individuals influential among our key audiences or stakeholders?

1. Does the incident directly affect our core business operations?

1. Have we encountered a similar incident in the past? In essence, is this a recurring issue for the company?

1. Is the incident linked to broader industry challenges or trends? Are competitors or others facing similar issues?

1. Are vital business pages currently accessible?

Incident Matrix

```
<table class="table table-striped table-bordered">
<thead>
<tr>
<th style="background:red">Level 1</th>
<th style="background:yellow">Level 2</th>
<th style="background:pink">Level 3</th>
</tr>
</thead>
<tbody>
<tr>
<td>High risk</td>
<td>Medium risk</td>
<td>Low risk</td>
</tr>
<tr>
<td>Leaked mission-critical keys + env variables</td>
<td>Mission-critical / legal content errors (ex: incorrect pricing, drastic typos or verbiage errors on our high converting pages)</td>
<td>Section of site is missing (ex: events, press releases)</td>
</tr>
<tr>
<td>Major vendor failures related to infrastructure (GCP + Contentful).</td>
<td>Integration failures (6sense, GA, etc)</td>
<td>Performance issues</td>
</tr>
<tr>
<td>Mission-critical pages are missing (ex: Homepage, missing, primary navigation)</td>
<td>Significant performance issues</td>
<td></td>
</tr>
<tr>
<td>See <a href="reporting-an-incident">reporting an incident below.</a></td>
<td>Create an issue and post in digital-experience Slack channel</td>
<td>Create an issue and post in digital-experience Slack channel</td>
</tr>
</tbody>
</table>
```

Reporting an incident

We now use incidents instead of using issues for documenting site outages. Incidents behave similarly to issues, allowing us to use templates tailored for outage documentation. This method provides more insights into our site reliability and ensures tracking and resolution of downtime events across our projects.

Point person: Nathan Dubord - Working hours: 9am - 6pm Eastern

1. Post in the digital-experience Slack channel and tag @digital-experience.

1. If there is no response within five minutes, please text or phone the following people:
 1. Eastern Timezone (UTC-5):
 1. Nathan Dubord
 2. Laura Duggan
 2. Central Timezone (UTC-6):
 1. Megan Filo
 3. Pacific Timezone (UTC-8):
 1. Javier Garcia
1. Incident created by DEX team member based on the project. For example, an outage in the Buyer Experience project would be created here. Note: Any time we would create an issue for an outage, create an incident instead. Make sure we open incidents in the appropriate project as this affects our reporting and metrics. As a general rule, an incident should be created if we are circumventing the triage process, there is no existing open issue, and our site uptime is affected.
 1. Consider filling the Severity, and Timeline Events when appropriate.
 1. After the incident is resolved, you may close the incident. This is what affects our time to resolve outage metric (Time to Resolve = Time Incident was closed - Time Incident was opened).

Call on the phone if no response within 15 minutes

PagerDuty

In the case of an emergency, Digital Experience Engineers have the ability to create a pager duty incident and trigger alerts on each others mobile devices. These pager duty incidents can also be triggered by GitLab team members with access to Pager Duty (IT Security, Reliability, SIRT, ect)

To trigger a pager duty incident for the Digital Experience team following these steps:

1. Report an incident by typing /pd trigger anywhere in Slack.
2. Select about.gitlab.com as the impacted service
3. Complete fields - priority, description, Urgency
4. This will create an incident and notify PagerDuty to alert members of the Digital Experience team.
5. PagerDuty will continuously escalate until contact with a team member has been made.

SECTION: marketing/digital-experience/engineering/localization.md

Purpose

Now that the Digital Experience team and the Localization team have collaborated for many quarters, we want to outline our learnings to work more efficiently together.

Existing Labels

dex-L10N: issues that involve both teams

L10N-attention: how to raise awareness to localization team to efforts that require their

insight

List of all labels in the Digital Experience group

Summary

Right now, an issue is created leading to UX and content to start working. Ideally, when the content is completed, UX can begin their work. After the UX is completed, this allows engineering and localization to begin their work. When engineers kick off a merge request, this allows content, UX, and engineering to make adjustments to the page. Note that we need the source language to be completed before translations can begin.

Challenges

All of this has been synthesized by the following issue comment as well as from Phase 1 of localizing content, which fall into two buckets:

1. Technology: The engineering review process, and integration with other tools
1. People: Project management, including flow and organization

What is Argo? How do I request access to Argo?

Argo is a translation service used by the localization team to manage files and assets that are to be translated. This epic was created to track onboarding into Argo. Here are issues that talk about assets and requests in the UI. You can request content that lives in YAML, or that exists in Contentful

When should pages become translated? Is it possible to release pages in multiple languages?

We are not in a mature enough spot to release in multiple languages. In our current state, we struggle between the increased lead time for translators, more complex QA processes across languages, and alignment between different teams.

As a rule, we are firing off translation requests after the page is live in English. This is to avoid back and forth between content and localization. This means that we cannot start translating a page until after the English page is released.

Contentful

What are source entries in Contentful?

This content comes from this issue, where we define source entries as a relationship between the authored content and the localized version. This is so that updates to the authored content can be correctly changed on the corresponding cloned entry. Whenever an entry in a different language is created, we must link it back to its English equivalent. This is important for the continuous translation process. This applies to entries created by engineering.

Every Content Type needs to have a single Entry Reference field so updates to the source

content are correctly put onto the corresponding translation Entry when translated. The reference field should only accept the same Content type as its source field, and it should be hidden from the edit view, to prevent users from accidentally changing it.

What is the difference between field-level and entry-level localization?

Field-level localization allows content in different locales to exist within the same entry, with each locale being a field of the entry. Entry-level localization is when every different localized content piece is used as unique net-new entries with each locale. One way to think about this is that field-level localization creates more fields, but fewer entries. Entry-level localization creates more entries, but fewer fields.

Per this comment, we typically use both field and entry-level localization. It is up to the localization team in collaboration with Digital Experience to see which approach makes the most sense. Right now, prioritize using field level over entry-level, as entry-level is a lot more complex both for the Argo integration and engineering.

Best practices for content editors adding localized content

Many of our webpages were built using a variety of standards, so some webpages will require a code change the first time they are localized. For this reason, if it's the first time a page is being translated, it's best if a DEX engineer reviews the page first. After any necessary code changes have been made and merged, content editors can edit translated content in Contentful. An exception to this rule is the Homepage and Pricing page - please make your edits then tag a DEX engineer to review and publish. In general, we avoid making changes on Fridays so as to avoid any potential bugs appearing on the weekend - if it's anything other than a small content change or typo, try to avoid publishing changes on Fridays.

Known issues: Since many content editors don't have access to Admin features in Contentful, they will not be able to bulk publish. This means that everytime the publish button is clicked, a new pipeline will begin. DEX engineers can help with bulk publishing to avoid this issue. If a change was published more than 30 minutes ago and it's still not visible on the website, please reach out to the digital-experience-team slack channel, so we can investigate if there was a failure.

Engineering

How Engineering kicks off a translation request

Engineers can kick off the translation process by creating a new issue using the localization template in the Localization Issue Tracker project. This eventually leads to an item being created in Argo.

The source language's content must be completed for this to go through. Given that it takes them between 5-7 days to translate, communicate that with your stakeholders before committing to work. As a general rule, content should be completed before the start of an iteration to give ample time for translations and LQA.

Note: we may need to kick off a translation request ourselves in Argo, but this part of the process is a work in progress.

Best practices when building marketing pages to support internationalization

If pages are built in Contentful, avoid custom JSON. This is important.

Custom JSON creates delays in the translation process, which slows down the velocity of our progress

Make sure to link source entries from different locales to their English version. This is crucial for the continuous translation process. If you create new entries in new locale, please link them to the source entry.

Encourage UX and L10n to talk to each other.

When new pages are created, we should use the i10n-attention label

Overflowing text due to the length of different character averages among locales

How do we deal with Marketo Forms?

Right now, each Marketo form ID is unique per every locale. We are looking into a potential solution so that one form holds each locale's content. This is being scoped out in this issue.

Global Gateway

The global gateway is crucial for creating a positive first impression by presenting content in users' native languages, fostering trust and ensuring seamless navigation. It enhances user experience by directing them to localized marketing sites and correctly localized product sign-up paths.

Our FY25 goal for the marketing site Global Gateway is to create a more personalized and effective marketing experience for users worldwide, ultimately driving better engagement and business outcomes. We will measure engagement and business outcomes with Google Analytics and Tableau.

Language Selector

The language selector is a feature implemented in the Navigation repository that allows users to choose their preferred language for viewing GitLab. Here's the `language.service.ts`

To display the available languages for a page, the Language Selector makes HTTP requests to check if the current page exists in each of the site's available languages. It sends one request per language, and if the request returns a 404, that language will not be displayed in the Language Selector.

After a user selects a language, the app stores the selection in the user's browser for use by the application's localization logic. We store it in `localStorage` here.

The language selector is the only place that can update the saved language in the user's browser.

Links Localization

Our application includes a Vue Mixin used across all pages. This Mixin scans each page for links that might be localized.

The Mixin checks the user's browser for their preferred language. If it finds a localized version of a link that exists, the localized version of the preferred language link will be statically generated.

Hreflang

We use the Hreflang HTML attribute to specify the language and regional targeting of our localized pages. It signals to search engines the language or regional variation of your content.

We use a Vue Mixin to generate our hreflang.

Common confusing case scenario

A common scenario that might be mistaken for a bug is when a user navigates to a localized page without using the language selector. For example, if the stored language is French (fr) and the user visits a Japanese page, the links inside that Japanese page will still redirect to the French version due to the user's preferred language setting.

SECTION: marketing/digital-experience/engineering/major-league-hacking-fellows.md

Program Information

The purpose of this page is to present process guidelines for the Major League Hacking Fellow cohorts (<https://fellowship.mlh.io/>) that are working with the Digital Experience team. This page is for fellows to find information about their cohort, the Digital Experience team, our projects, and where to ask for help.

Fall 2022 Cohort

The Fall 2022 Cohort working with GitLab will be comprised of 2 engineering fellows. They will be making open source contributions to the Slippers Design System.

- 9/19/22 - start - National Butterscotch Pudding Day
- 1st week - orientation, team building, onboarding
- 12/16/22 - end date (12 weeks)

Improving the Slipper Design System at GitLab

Summary of the Project

At GitLab, the Slippers Design System is a systematic approach to marketing site development. You can use the system to manage our marketing component library.

In this project, the MLH fellows will complete several deliverables that will improve the user experience of the Slippers Design System. We will implement new components, iterate on existing components, and use UI components from GitLab's Pajamas design system to bring consistency across the GitLab application and marketing site.

Stakeholder Information

Please list all stakeholders (name, email, title/role) who will be involved with the project.

Name	Role	GitLab Handle
-----	-----	-----
Lauren Barker	Maintainer	@laurenbarker
Megan Filo	Maintainer	@meganfilo
Gideon Tong	MLHF Pod Leader	@gideontong
Chukwuemeka Mba	MLHF Fellow	@Chukwuemeka-Mba
Uy Seng	MLHF Fellow	@nova44056
Seung Yoo	MLHF Fellow	@SssngM

Fellow Technical Onboarding

Slippers is the open-source GitLab Marketing Web Design System. It was created in the spirit of "everyone can contribute". Guidelines enable people to build better web pages and co-create with ease.

To get started with contributing to the Slippers Design System, create a free SaaS account with GitLab.

Watch this video on how it's currently built, published, and used on the marketing site as of August 2022.

Create a fork of the Slippers repository.

Follow the README instructions for setting your local development environment.

Tutorials/Videos

- Create a GitLab account and set up SSH keys.
- <https://gitlab.com/gitlab-com/marketing/digital-experience/slippers-ui>
- If you don't know VueJS 2.0, you can get started on this guide
- Slippers Design System walkthrough video

Documentation

- Read GitLab's contribution guidelines with a special focus on the Merge Request Workflow
- Get familiarized with GitLab's Slipper Design System by reading the intro and exploring our Storybook instance.

Deliverables + Timeline

Please identify and describe the deliverables for your project(s) and the ideal deadline for each. This may include features or issues you would like the fellows to complete by the end of

the fellowship. There are two general approaches we've seen be successful:

Issue	
Description	
Assignee	Deadline

----- -----	
· Vertically align dropdown caret	Vertically align dropdown caret
Seung Yoo	October, 17th
· Loading components	Create progress bar and spinner loading
components	Uy Seng (Wei)
October, 17th	
· Address v::deep errors on build on Slippers repo	Update v::deep references that are now
deprecated	Uy Seng (Wei)
October, 17th	
· Add Slippers variable names next to colors	Add the matching Slippers variable
within variables.scss to the table on our color page	Uy Seng (Wei)
October, 17th	
· Add Slippers variable names next to spacing	Add the matching Slippers variable
within variables.scss to the table on our spacing page	Uy Seng (Wei)
October, 17th	
· Create tooltip component	Create a tooltip component in the design
system	Chukwuemeka Mba November, 17th
· ENG: Accessible and compliant dropdowns	Iterate on dropdown component to be
accessible	Uy Seng (Wei)
November, 17th	
· Implement the Storybook/Figma plugin closed with notes	Implement the
Storybook/Figma plugin and make it available for use in the project	Chukwuemeka
Mba November, 17th	
· Update Slippers Intro/Landing page closed with notes	Improve our Slippers
Design System landing page	Chukwuemeka Mba
November, 17th	
· Create Component Overview page	Improve our UX by creating a component
overview page	Uy Seng (Wei)
November, 17th	
· Accordion component	Build the accordion component as
designed in Figma into Slippers.	Seung Yoo
November, 17th	
· Update card component - variant 1	Update our card component styles and folder structure
Uy Seng (Wei) December 8th	
· Add Footer to each page closed with notes	Investigate how to and add a footer to each
page in Storybook Uy Seng (Wei) December 8th	
· Update breadcrumb component	Update existing breadcrumb component Uy Seng (Wei)
December 8th	
· Update Top Button name to be Side Button	Update name of Top Button component Seung Yoo
December 8th	
· Update RadioGroup component functionality and states	Update existing RadioGroup component
Seung Yoo December 8th	

· Add checkbox components to Slippers + create new Controls folder	Create new component and update folder structure	Uy Seng (Wei)	December 8th
· Update horizontal rule component color	Update HorizontalRule component to use new brand color	Seung Yoo	December 8th
· Change CustomDropdown component placeholder text	Change placeholder text in CustomDropdown story file	Chukwuemeka Mba	December 8th
· Add mobile styles for Quote typography variables	Add new font styles on mobile in variables file and update SlpTypography component	TBD	December 8th
· CustomDropdown component MVC2	Second iteration to our CustomDropdown component	Chukwuemeka Mba	December 8th
· Update tooltip component arrow size	Implement design feedback from MVC1 review	Chukwuemeka Mba	December 8th
More issues if there's capacity from fellows			
TBD			
TBD	December 8th		

MLH Partner Success Contact

- Matt Dillabough

SECTION: marketing/digital-experience/engineering/marketing-site-approval-process.md

Going forward, all changes on the marketing site (about.gitlab.com) must follow an approval process prior to merging in changes on the website.

Executive Summary

The lack of an approval process for changes going to production on GitLab's Marketing site creates a risk for our business because, as it stands today, anyone can push a change live to our site at any time (details of risk outlined in the Problem Statement below). We are introducing a review process for any contributions going live on the Marketing Site.

Problem Statement

GitLab's Marketing site has undergone significant changes over the last two years. GitLab's Marketing team has collaborated on a strategic user flow and is working on specific user journeys that drive our business metrics. As such, we require approvals and controls in place to ensure that all contributions to the Marketing site support ongoing efforts. There are currently no controls or permissions in place, which creates a risk to our business performance. Risk in this situation can take the form of broken analytics data, inconsistent brand messaging, negative impact on SEO ranking, and potential legal issues.

Symptoms

At its peak, GitLab's Marketing site contained over 3500 pages. Many of these pages receive little to no traffic and have not been updated or deleted since they were created. One can

assume the root cause for this bloat is a lack of accountability due to the lack of approvals to ensure content is added or modified with intent and coordination in collaboration with cross-functional teams.

Another example is cases where we have several similar pages that may make sense from an inside-out perspective, but from a customer-centric perspective, they confuse our prospective customers. Example: CI/CD Pages:

- 1. <<https://about.gitlab.com/solutions/continuous-integration/>>
- 1. <<https://about.gitlab.com/topics/ci-cd/>>
- 1. <<https://about.gitlab.com/resources/ebook-single-app-cicd/>>
- 1. <<https://about.gitlab.com/webcast/mastering-ci-cd/>>

A byproduct of the uncontrolled increase in pages is the unintended effect of diluting the value of the data we collect from Google Analytics. With so many pages receiving little traffic and providing little value to our prospective customers, we struggle to get concentrated data on core pages.

The codeowners feature was launched on GitLab in 2019, and this requirement was identified with support from the previous CMO but was never actioned: <https://gitlab.com/gitlab-com/www-gitlab-com/-/issues/4085>

Goals

The goal of putting controls and approvals in place is to ensure that all changes to the GitLab Marketing site (about.gitlab.com, not including handbook.gitlab.com/ or about.gitlab.com/direction/) support the defined objectives of GitLab's Marketing team. We will operate the same way as our Product team in that the appropriate people approve all MRs before going into production.

Hypothesis

The approval process will increase the consistency of our Marketing site while ensuring ongoing initiatives are not at risk of being disrupted due to unapproved changes going live to production. The approval process will also increase visibility and collaboration to provide a customer-centric and coordinated effort for those looking to improve GitLab's Marketing site.

Strategy

We do not want to compromise GitLab's everyone can contribute mission. Our strategy is to implement a minimal approvals process. Currently, an MR goes live straight to production; this change results in a contribution taking the form of an AB Test proposal (issue) or for an MR to be reviewed before being merged into production.

The Digital Experience team has set up a project for AB Test proposals as a repo: <https://gitlab.com/gitlab-com/marketing/digital-experience/ab-testing>. Anyone can contribute an issue using the AB Testing issue template (template in progress).

Not everything needs to be an AB Test proposal; we still prioritize MRs as defined in the handbook: everything starts with an MR.

We will have four approval flows, from minor to major change. The goal is for a member of GitLab's Digital Experience team always to review an MR before it goes live on the Buyer Experience repo or key marketing pages in the www.gitlab.com repo.

Approval Levels

1. Level 01 - Minor Change

- 1. Label: dex-approval::1-minor
- 1. Example of the type of change: typos or content changes.
 - 1. Anyone on the Digital Experience team can approve and merge.
 - 1. Once approved by the Digital Experience team, the MR creator can merge.
- 1. Example of the type of change: /customers filter is displaying the wrong results.
 - 1. Anyone on the Digital Experience team can approve and merge.
 - 1. Once approved by the Digital Experience team, the MR creator can merge.

1. Level 02 - Standard Change

- 1. Label: dex-approval::2-standard
- 1. Example of the type of change: re-arranging components on a page.
 - 1. Once the Digital Experience team approves, members of the Digital Experience team can merge.
 - 1. Members of the Digital Experience team should keep their Managers informed.
 - 1. Manager's discretion to align with the Director of Digital Experience or not.
- 1. Example of the type of change: Adding new pages.
 - 1. Managers of Digital Experience can approve and merge.
 - 1. Manager should align with Director, Digital Experience.

1. Level 03 - Business Impact Change

- 1. Label: dex-approval::3-key-page
- 1. Example of the type of change: Changing Nav
 - 1. Directors and Managers of Digital Experience need to align.
 - 1. Managers of Digital Experience can approve and merge.
- 1. Example of the type of change: Changes to key drivers of traffic and/or business goals etc. (Homepage, Pricing Page, Why GitLab).
 - 1. Director, Digital Experience can approve and merge.
 - 1. Director, Digital Experience will keep Marketing leadership informed of all Major Changes.

1. Level 04 - Legal Change

- 1. Label: dex-approval::4-legal
- 1. Example of the type of change: Any changes to company information, board/director information, etc.
 - 1. Legal and Director of Digital Experience must approve.
- 1. Example of the type of change: Changes to sign-up workflows or alteration/omission of footers/links to anything legal-related.
 - 1. Legal and Director of Digital Experience must approve.

Once an MR is ready for review, approval outlined in the above sequence must occur to merge an MR into production.

All pages of the Marketing site (about.gitlab.com minus /handbook) are included in the requirement of this new approval process.

Steps to get Approval

When creating a merge request in the Buyer Experience Repository (where we are migrating our marketing pages), please use the marketing-site-change template.

Step 1: What is changing in this MR?

Please describe the change and link any relevant issues.

Step 2: Add the appropriate labels for triage

The MR will have dex-approval::2-standard automatically applied, but please update it to one of the following, based to the approval levels above.

- 1. dex-approval::1-minor
- 1. dex-approval::2-standard
- 1. dex-approval::3-key-page
- 1. dex-approval::4-legal

Step 3: Tag the appropriate people for review

Depending on which label is used, you may tag the following people as a Reviewer on this MR:

- 1. Level 1: Any member of the Digital Experience Team
- 1. Level 2: Any member of the Digital Experience team
- 1. Level 3: Ping @gitlab-com/marketing/digital-experience in a comment. This will tag the Digital Experience Leadership team using @gitlab-com/marketing/digital-experience, and they can review. When in doubt, tag @mpreuss as a reviewer.
- 1. Level 4: This will need Legal approval. Tag @mpreuss and he can loop in the legal team.

For changes to Support-related pages, please tag @jcolyer for review.

SECTION: marketing/digital-experience/engineering/marketing-site-rearchitecture-project.md

Why We Are Proposing This Change

Over the course of FY21 and FY22 it's been challenging to prioritize and make progress on the Monorepo Project. The lack of improvement to the developer experience is compromising the Digital Experience team's efficiency and we're not able to deliver our team's true output.

GitLab's Marketing site is also experiencing several technical issues that affect our current and prospective customers due to the complexities of our current repo. Digital Experience is not able to effectively solve and release fixes quickly which is undoubtedly causing GitLab to lose potential customers.

After considering 3 different approaches we've aligned on the following as our best option.

What We Are Proposing

- 1. Making key business driving pages of GitLab's Marketing site build faster, resulting in

increased engineering output. Anticipated ROI includes:

1. Pipeline times: currently we are at about 20 minutes average pipeline time. There are some work items that we need to see in the review app (anything requiring a data file), which means it can take 20 minutes to validate a change.

1. Merge train times: since we work in a shared repository with the handbook that receives so many MRs so often, our work ends up in long merge trains. Looking at a recent production deploy on 2021-07-23, this MR took closer to 40 minutes to go live.

1. Development feedback loop: running the Middleman preview server (our local development environment) by itself can take 3 full minutes to reload changes to our codebase. We implemented a monkey patch on Middleman to cut that down, and reduced it to 46 seconds. Benchmarks here. But the monkey patch doesn't work on data-driven routes, so as we move more and more content into Netlify CMS, we spend more and more time waiting on three-minute refreshes in development.

1. Decreasing size of Marketing site repository to ensure stability

1. As of 2021-07-23 the files in our repository take up 2.6GB of space. But GitLab registers it as over 68TB of storage overall.

1. Connecting Storybook design system platform with Marketing site to increase design & engineering output

1. Current state: engineers build a single component twice. Once in vue.js in Slippers, and one in www-gitlab-com in ruby

- Future state: engineers build a Vue.js single component in slippers. This component can be directly used in Nuxt.js

What We Are Not Proposing

1. Changing anything to the current processes of the Handbook

1. Changing any processes to the pages not identified as part of this project

Goals and Measurement

Ultimately this plan will reduce the time, money, and resources required to meet our team goals. Specifically we will look to measure:

1. Reduce the time it takes to push a change to production

1. Local build time

1. Pipeline time

1. Increase the frequency of MRs

1. Decrease time to close issues

1. Decrease negative effects on Infrastructure and SRE team

Pages We're Focusing On First

We plan on starting with important pages that are simpler with lower risk, gradually increasing in value and risk.

1. Enterprise

2. SMB

3. Free Trial

4. Pricing

5. Homepage

Scope

- 6-8 weeks with 2 engineers
- Targeting 5 pages fully transitioned with the remaining page listed below in Scope being completed through Q4

Roles

DRI: @laurenbarker

Team Members Involved and Why

- @laurenbarker: Senior Fullstack Engineer who is most familiar with our tech stack, needs, and requirements.
- @tywilliams: Excellent engineering (specifically Ruby) skills required for this project.
- cwoolley-gitlab: Chad will be a stable counterpart and consult in this effort. Chad's involvement to date as well as his knowledge will greatly increase the success of this project.
- @mpreuss22: Product Management. Digital Experience team leader, accountable for successful delivery of this effort.

Needs

We will require GCP Deployment Keys from Infrastructure. We would also like a new storage container to enable the plan we have for the new build process.

Scope of Needs

1. Move 10 - 25 marketing webpage to new static site generator repository
 1. Pricing
 1. Homepage
 1. Free Trial
 1. Sales
 1. Demo
 1. Install
 1. Features (Landing and single)
 1. Topics (Landing and single)
 1. Solutions (Landing and single)
 1. Enterprise
 1. SMB
 1. Partners (Landing and single)
 1. Comparison Pages
1. Switch to Nuxt.js as SSG
1. Seamless integration with Slippers for efficiency and developer experience
1. Static site generator builds and deploys these 10 - 25 pages to the existing about.gitlab.com domain
1. Digital Experience team reduces maintainance of the www.gitlab-com repository
1. Iteratively migrate parts of marketing site to new repo over FY23
1. Buyer Experience Repository

Benefits

- 1. Increased output on key business driving page
- 1. Minimizes breakage for pages like the release post
- 1. Sets up a migration plan we can continue to work on as new pages become prioritized

Risks

- 1. Will need very clear documentation to ensure clarity of what lives where
- 1. We will most likely need support from Infrastructure
- 1. Duplication of dependencies on both sites (until entire Marketing site is moved to new architecture)
- 1. Negative impact to other teams' workflows as we move the single source of truth for some pages elsewhere.
- 1. Time estimation on related tasks will be inconsistent and incorrect as we learn what to anticipate with a new codebase.

Definitions

- 1. Core Marketing Site: Key pages that drive ARR
- 1. ARR: <https://handbook.gitlab.com/handbook/sales/sales-term-glossary/arr-in-practice/>
- 1. Vue.js: <https://vuejs.org>
- 1. SSG: Static Site Generator
- 1. Nuxt.js: <https://nuxt.com/>

SECTION: marketing/digital-experience/engineering/navigation-repository.md

The navigation repository (also known as be-navigation) is a separate package that is updated and maintained independently from the rest of the marketing website. This is so that we can make changes in one place, and have any consuming repositories pull from that single source of truth. The navigation is currently maintained by the Digital Experience team.

Navigation is following Semantic Versioning. The current released version can be found on this [npm](#) page under Versions.

Making Changes

Navigation data is distinct from the Navigation code, as it originates from Contentful. To effect changes, two methods are available, contingent on the nature of the desired alteration.

Content Changes

Navigation data is stored within Contentful, and it is utilized by projects that rely on the Navigation repository. Subsequently, this data is transferred to the Navigation component.

Below, you will find two entries containing the data for the navigation in Contentful.

1. Navigation Entry
2. Footer Entry

Localization data is managed within this context, where child entries are organized by language, and each entry is named with its respective language code format, such as [fr-fr] - Name of the entry.

To ensure that changes become visible in production, it's necessary to publish the data in Contentful. Afterward, within a few minutes, the updated data will become accessible on the website once the pipeline of the corresponding project has been successfully executed.

Structural Changes

To request changes to the Navigation repository, please fill out an issue here to be triaged by the Digital Experience team.

If you would like to self-serve changes to the navigation, feel free to clone the project locally following the instructions in the readme. Once you make changes, please tag a member of the Digital Experience team to review your MR. We are consistently running tests on navigation links, and doing research and design spikes on navigation changes, so all changes will need to be approved by a member of our team.

When making a change to the navigation, it's important to check for an AB test. The quickest check is to look at the file system for the navigation repository. Is there a folder for Navigation as well as a folder for NavigationB (or some other duplicate name that contains similar files)? If so, be sure to make your changes in both folders, so that both variants will see the updates.

Note: Two JSON files are stored under a folder named mocks. The data within this folder is exclusively for testing purposes, and any modifications made will have no impact on the production data.

Releases

Currently, our team releases new navigation changes in bundles on Wednesdays. This can be adjusted as necessary, and we typically opt to release large, breaking changes in their own version.

We avoid deploying minor or major releases of the navigation on release post days.

Releasing involves 4 main steps

1. Prepping issues for the release
2. Publishing a new version to npm
3. Updating any repositories that consume the navigation to point to the new version
4. Publishing any changes made in Contentful

Detailed instructions

1. Create a new issue titled Navigation release: Version x.x.xx and include links to the MRs and Issues included in this release. This navigation issue should be added to the quarterly navigation release epic (ex. FY23Q3 epic). If this has been assigned to you already, you can skip this step.
1. Create a new branch from main, with all changes
1. Increment the version number in package.json
1. Build /dist folder - yarn build (optionally use yarn link for a detailed local review)
1. Publish to npm - npm publish (Note: You may need to login with your npm credentials using npm login)
1. Merge changes to origin/main
1. Once the package has been published it's now ready to be updated in the consuming repositories. This should include the Buyer Experience and www repositories. Open an MR in those repos using the navigation-change description template. Include links to those MRs in your Navigation Release Issue assigned